Z Empire Auto Accessories Shop Management System

System Development Report

An ASP.NET Core 8 MVC application for an automotive accessories, window tint, paint protection film and ceramic coating business

Repository	SeanEsperancilla/ZEmpireAutoAccessories
Branch	Programmer
Head commit	cdd396d
Development period	20 August 2026 – 10 October 2026
Commits on branch	188
Database	Microsoft SQL Server — ZEmpire
Report compiled	23 October 2026
Edition	Second edition — supersedes the edition of 2 October 2026

Contents

1 Executive Summary
2 System Overview
3 Technology Stack
4 Architecture and Design
5 Security and Access Control
6 Functional Modules
7 Core Domain Logic
8 Presentation and the Design System
9 Verification and Quality Assurance
10 Defect Log
11 Data Management and Deployment
12 Development Chronology
13 Project Metrics
14 Limitations and Future Work
A Appendix A — Complete Commit History
B Appendix B — Repository Attribution

1 Executive Summary

This report documents the design, construction and verification of a shop management system for Z Empire Auto Accessories, a Philippine automotive shop whose trade is window tinting, paint protection film (PPF), ceramic coating, detailing and the supply and fitting of tonneau covers and related accessories.

The system replaces manual record keeping across the whole commercial cycle: a customer and their vehicle are registered, the car’s condition is recorded at intake, a quotation is raised, converted to a job order, carried through to a service invoice or a direct sale, settled at a cashiering screen, evidenced with proof of payment, covered by a warranty, and reported on. Behind that cycle it maintains a perpetual inventory in which film is deducted from the roll by the actual length each panel of glass consumes, rather than by a nominal line quantity.

The application is an ASP.NET Core 8 MVC solution of approximately 29,198 lines of C# and Razor across 24 controllers, 15 service classes and 108 views, mapped by Entity Framework Core onto an existing Microsoft SQL Server database of 38 entities organised into seven schemas. It was built over 188 commits between 20 August and 10 October 2026.

Two constraints shaped the engineering throughout and are the source of most of what is technically interesting in this report. **The first is that the database schema could not be altered**: the database was in live use, and an early attempt to add a column caused a production failure and was reverted. Every subsequent feature that needed to record something the schema had no column for had to find another home for it. **The second is that the database’s own CHECK constraints are authoritative**, so application validation was derived from them rather than invented alongside them.

The response to the first constraint is a recurring pattern described in section 4.4: data the schema cannot hold is kept in version-controlled JSON files beside the application and loaded at startup, with the application’s existing abstractions reading from them transparently. This pattern carries the cut-size table, the per-product unit of measure, and the stock posting ledger, and it allowed three substantial features to be delivered without a single DDL statement.

A second, quieter technique proved as valuable: **several columns already present in the schema had never been written to by any screen.** The shade recorded against a document line, the shade breakdown on a stock count sheet, and the per-shade stock estimate derived from the two (section 7.12) were all delivered by filling columns the database had carried unused since it was designed, rather than by adding anything. Auditing the schema for what it already allowed turned out to be more productive than asking what it lacked.

A third theme emerged late and is documented in section 8. Correctness had been pursued screen by screen, and the presentation had drifted the same way: ninety of the views carried a private copy of the same stylesheet, and no two screens quite matched. Consolidating them into one stylesheet removed 2,238 lines of duplicated CSS and, in doing so, exposed a layout defect that had been clipping values on sixteen screens. **A system whose look is maintained in ninety places has ninety places for a defect to hide.**

2 System Overview

2.1 Business context

Z Empire Auto Accessories sells and fits products that are measured in two fundamentally different ways. A tonneau cover, a horn or a bottle of shampoo is a discrete item: one is sold, one leaves the shelf. Window tint and paint protection film arrive on rolls and are consumed by length, and the length consumed depends on which panel of which class of vehicle is being covered. A front windshield on a van is not a front windshield on a sedan.

Pricing follows the same split. Accessories carry a single default price. Film and coatings are priced on a four-way matrix of product, tint variant, vehicle classification and panel, so that a Profilm X Diamond coating on a van and the same coating on a small sedan are different line items at different prices. The system had to represent both models without forcing either into the other’s shape.

2.2 Scope

The system covers customer and vehicle records; a product and service catalogue with its pricing matrix and price history; quotations, job orders and vehicle checklists; service invoices and direct sales; cashiering and payment capture with proof of payment; warranty registration and claims; perpetual inventory with stock counts, panel-based deduction and a per-shade estimate; reporting; and user, role and module-permission administration.

3 Technology Stack

Layer	Technology	Notes
Runtime	ASP.NET Core 8 (.NET 8)	Long-term support release
Pattern	MVC with Razor views	Server-rendered; no SPA framework
Data access	Entity Framework Core 8	Database-first; no migrations (see 4.2)
Database	Microsoft SQL Server	Database name ZEmpire; seven schemas
Authentication	ASP.NET Core Identity	Tables mapped into the asp schema
Authorisation	Claims-based, custom attribute	ModuleAuthorizeAttribute over sec.RolePermission
Front end	Bootstrap 5, vanilla JavaScript	No client framework or build step
Styling	One shared stylesheet	site.css carries the page furniture (see 8)
Document output	QuestPDF	Receipts, invoices, checklists and report exports
Configuration	appsettings.json	JSON-with-comments; bound at startup
Side-file storage	JSON under App_Data	Schema-free extension (see 4.4)
Source control	Git / GitHub	Feature work on the Programmer branch

4 Architecture and Design

4.1 Layering

The application follows a conventional three-layer MVC arrangement. Controllers handle HTTP concerns, model binding and view selection. A service layer behind thirteen interfaces holds the domain logic that more than one controller needs or that is too involved to sit in a controller action — pricing resolution, inventory movement, quotation conversion, cashiering totals, reporting aggregation, document generation, stock posting and the per-shade stock estimate. Entity Framework Core maps the model classes onto the existing SQL Server tables.

Services are registered in the dependency-injection container at startup and injected by interface. Three are registered as singletons rather than scoped because their purpose is an in-memory cache over a file on disk: the cut-size store, the stock posting log and the measure store. Each guards its cache with a lock and replaces it whole on write, so a reader never observes a partially applied update.

4.2 Database-first, and the absence of migrations

The database existed before the application and remained in live use throughout development. The model classes were therefore written to match the database rather than the database being generated from the model, and EF Core migrations were deliberately not used. The practical consequence is that **the model is a description of an external fact, and any disagreement between the two is a defect in the model.**

Two defects of exactly this kind were found and corrected during a re-audit of the model against the live database, and a third — a NULL-materialisation crash that occurred whenever a customer record carried a NULL in a column the model declared non-nullable — affected every list that included a customer and was fixed in three places at once. A fourth, documented as defect 17, reproduced the same failure on vehicles: adding a

validation attribute to a property silently changed how EF Core maps the column.

4.3 Schema organisation

Schema	Holds
asp	ASP.NET Identity — users, roles, claims, logins, tokens
cat	Catalogue — products, categories, tint variants, shades, panels, pricing, services
crm	Customers, vehicles and vehicle classifications
inv	Inventory transactions and stock counts
ops	Operations — quotations, job orders, job types, vehicle checklists, warranties
sales	Direct sales and their line items
sec	Module definitions and role-to-module permissions

Stock on hand is not a stored column. It is derived as the sum of inbound transactions less the sum of outbound transactions over inv.InventoryTransaction, mirroring the database's own dbo.vw_StockOnHand view. This makes the inventory a **ledger**: every movement is a row with a cause, and the balance is always reconstructible from history rather than being a number that can silently drift.

4.4 The schema-stability constraint and the side-file pattern

The governing constraint of the project was that the database schema must not change. This was not a preference. An early change that added a column to the product table caused the live application to fail on startup with an invalid-column error, and was reverted in full. From that point the schema was treated as fixed.

Three features nonetheless needed to record facts the schema had no column for:

- **Cut sizes** — how much film each panel takes on each class of vehicle. The pricing table is keyed on exactly product, variant, classification and panel, but carries only a price.
- **Unit of measure per product** — whether a particular product is measured off a roll or counted in pieces, where that differs from what its category implies.
- **Stock postings** — precisely what quantity each completed document took off the shelf, so that reversing it returns the same quantity.

Each is held in a JSON file under App_Data, read once into a cached singleton at startup, and surfaced through the abstractions the rest of the application already used — so call sites did not need to know where the answer came from. Writes are made by serialising the whole file to a temporary path and moving it into place, so a process that dies mid-write leaves the previous file intact rather than a truncated one that would read as 'nothing configured'.

Two of the three files are committed to the repository, because they are shop configuration that every installation needs. The third, the stock posting log, is deliberately excluded: it changes on every document completion and tracking it would produce a merge conflict on nearly every commit. Its absence degrades gracefully — reversal falls back to recomputing from the cut sizes, which is what the application did before the file existed.

> ***Design note.*** *Because these files are keyed by the database's own identifiers, they are only meaningful against the database they were written from. The deployment runbook therefore requires the database backup and the JSON files to be refreshed together; this is stated explicitly in the repository documentation.*

5 Security and Access Control

5.1 Authentication

Authentication uses ASP.NET Core Identity with its tables mapped into the asp schema. Two roles exist, Admin and Staff; an earlier Manager role was removed as redundant. A startup seeder ensures both roles exist and gives accounts seeded directly in SQL a usable password, so a fresh installation can be logged into without manual intervention.

Employee creation was changed early on to create a real login account as part of the same operation, rather than requiring an account to have been created beforehand, and to reject a duplicate employee number. Self-service password change and administrator password reset were added, both with show/hide toggles.

5.2 Module-level authorisation

Authorisation is not role-based at the controller. Roles are mapped to modules through sec.Module and sec.RolePermission, those permissions are projected into the signed-in user's claims as AppClaims.ModuleAccess, and controllers carry a custom ModuleAuthorizeAttribute naming the module or modules that grant entry. Where an attribute names more than one module the semantics are *any-of*: the Cashiering screens, for instance, admit a user who holds either Sales or Service Invoices.

This indirection is what allows the permission matrix to be edited at runtime from the Roles and Permissions screen without redeploying. It also meant that features added late in the project — the Cut Sizes grid, the Product Categories screen and the Shade Stock screen — could be given sensible access control by attaching them to the existing Inventory and Products modules, rather than requiring a new row in sec.Module, which would have been a data change to a live database.

The same reasoning governs cross-module links. The job order screen offers a way into the vehicle checklist (section 7.13), and that link is rendered only for a user who actually holds the Vehicle Checklist module, so nobody is shown a button that answers with a 403. The dashboard's quick actions are gated the same way.

A specific guard prevents an administrator from removing their own role's access to the module permission screen, which would otherwise lock every administrator out of the system irrecoverably through the user interface.

5.3 Authority over prices

Catalogue prices are an administrator's to set. A member of staff who changes one, deliberately or by accident, moves what every later quotation, job order and sale is charged at, and there is nothing on the resulting documents to show it happened.

Two surfaces carry a price and they were not equally protected. Changes to the pricing matrix have always been journaled to cat.PriceHistory with the old price, the new price, the user and the timestamp. A product's or service's default price is not journaled at all — there is no table for it and adding one is a schema change — so a change there left nothing whatever to look back at. **That is the surface most in need of control, and it was the one with none.**

Price editing is now restricted to the Admin role on both. The write actions carry an AdminOnlyPrice filter, and the controllers additionally discard a posted price from anyone who may not set one, because a field rendered read-only still posts its value and a form can be replayed. The forms show the field greyed with a line explaining why, rather than failing silently, and the write buttons are hidden rather than left to answer with a 403.

This is deliberately separate from the module permission system. Module access decides which screens somebody can reach; staff still need to look prices up, and still need to type a different price on an individual line where a quoted figure or a discount differs from the book. The restriction governs only what the book itself says.

5.4 Handling of evidence files and personal data

Proof-of-payment and warranty-claim attachments are customers' bank receipts and screenshots, carrying names, reference numbers and account details. They are stored under App_Data, outside the web root, so they cannot be retrieved by guessing a URL; they are served only through a controller action that runs the same module authorisation as the rest of the application. File names are constructed solely from the record type and its identifier, so a user-supplied name can never influence the path written to.

Uploads are constrained in three ways: an extension allow-list, a magic-byte check of the file's actual leading bytes, and a five-megabyte ceiling. The files are excluded from the Git repository by .gitignore, so sharing the repository with a teammate does not distribute customers' banking documents. A teammate's records display as missing their proof, which is the correct outcome — they genuinely do not have the file.

During the project the repository itself was made private, and a committed database backup was reviewed to confirm it carried only seeded demonstration data rather than live customer records before it was shared.

6 Functional Modules

Sixteen modules are defined in the permission matrix. Four further screens — Cashiering, Cut Sizes, Product Categories and Shade Stock — are full features that deliberately share an existing module's permission rather than defining their own, for the reason given in section 5.2.

Module	Responsibility
Customers	Customer records with an enforced 11-digit mobile number beginning 09; deletion blocked while vehicles are registered.
Vehicles	Vehicles against a customer, with classification, plate number and checklist history. Plate numbers are mandatory, unique and validated against Philippine car formats.
Products	Catalogue items, their category, default price and active flag; a Categories screen for the shelves themselves; per-product unit of measure.
Services	Labour and service items with their own categories and default prices.
Pricing	The four-way price matrix of product, tint variant, vehicle classification and panel, grouped and searchable on all four.
Price History	The record of price changes over time.
Quotation	Customer quotations and their line items, reduced to two states: Draft, or converted to a job order.
Job Orders	Work orders, converted from a quotation or raised directly; line items priced from the matrix; completion moves stock; a link through to the vehicle checklist.
Job Types	The categories of work a job order can be raised for.
Vehicle Checklist	The condition of a car as it arrived, panel by panel, captured in one form, summarised on the list, and issued to the customer as a signed PDF (7.13).
Service Invoices	Invoices raised from a job order or directly; payment capture with change; VAT breakdown; PDF receipt.
Sales	Direct over-the-counter sales with the same payment capture, stock warnings and receipt.
Warranty	Warranty registration against a sale or invoice line, expiry handling, and evidence of claim.
Inventory	Perpetual stock as a ledger, deliveries, stock counts, the Cut Sizes grid and the Shade Stock estimate.
Reports	Sales, collections, inventory and service-invoice reporting with PDF export.
User Management	Employees, their login accounts, roles, and the role-to-module permission matrix.

7 Core Domain Logic

This section covers the parts of the system where the problem was not the mechanics of a CRUD screen but a question of correctness in the domain. These are the areas where a wrong answer is **silent** — it produces a plausible number that is not checked until a stock count or an audit.

7.1 Value-added tax

Philippine VAT is 12% and is *inclusive*: the price quoted to the customer is the price they pay, and the tax is a component of it rather than an addition to it. The direct-sales table has no tax column, so VAT is derived for display and never stored.

The breakdown is computed by taking the vatable sales as the total divided by 1.12, rounded to two decimal places half-away-from-zero, and the VAT amount as the remainder. Taking the remainder rather than computing both halves independently guarantees the two always sum exactly to the total, with no centavo appearing or vanishing.

A rounding defect, and how it was found

The browser displays a live VAT breakdown as the cashier types. A verification run comparing the browser's figure against the server's across every total found disagreement on roughly two per cent of values. The cause is that JavaScript has no decimal type: a total of 6.30 divided by 1.12 is exactly 5.625 in decimal arithmetic and rounds up, but in IEEE-754 double arithmetic it lands fractionally below and rounds down. The browser calculation was rewritten to work in integer centavos, computing cents × 25 / 28 with explicit half-away-from-zero rounding. A re-run across 54,055 totals produced zero disagreements.

7.2 Payment, change, and the database as the authority

Analysis of the database backup established the exact text of the constraints governing service invoices. Two are decisive:

Constraint	Effect
CK_ServiceInvoice_Math	TotalAmount = (SubTotal − DiscountAmount) + TaxAmount, and ChangeAmount = AmountPaid − TotalAmount
CK_ServiceInvoice_Change	ChangeAmount >= 0, which means a short payment can never be stored at all

The second constraint is a business rule expressed in the schema: this shop does not record partial payment on an invoice. The application was therefore built to refuse a short payment at the form with a clear message, rather than allowing it through to be rejected by SQL Server as an unhandled exception.

An earlier defect in this area is instructive. When exact-amount settlement was introduced for non-cash payment modes, the clamp that set the tendered amount equal to the invoice total ran *before* the short-payment guard, which made the guard unreachable for every mode except cash — a 2,000 peso transfer against a 2,500 peso invoice was recorded as paid in full. The fix was to clamp only upward, so the guard still sees and rejects a genuine shortfall.

7.3 Payment modes and proof of payment

The shop offers four payment modes: Bank Transfers, Credit Cards, GCash and Cash. These are driven from configuration through a central policy class, with a startup seeder that renames legacy rows in place, inserts any missing mode, and deletes retired modes only where no sale or invoice references them, logging by name any it had to keep.

Two properties of a payment mode are tracked separately, and the separation was arrived at by correction. An initial design assumed one list could decide both whether a mode settles at an exact amount and whether it requires an attached receipt. Credit Cards broke that assumption — they settle exactly but initially required no slip — and the two concerns were split. The underlying rule turned out not to be 'online versus offline' at all, but simply whether there is anything to attach: only cash has nothing. Attachments can be removed and replaced

when the wrong file is uploaded, and the same mechanism carries evidence that a warranty has been claimed.

7.4 Units of measure

Roll goods are held in **centimetres** throughout. The inventory transaction table has no unit column, so a single base unit is the only way a running total means anything, and centimetres were chosen because every conversion the shop uses is exact in decimal arithmetic: one inch is 2.54 cm and one metre is 100 cm, with no repeating fraction. Staff enter quantities in whichever unit suits — a supplier delivers in metres, a cutter works in inches — and the conversion happens once, at the point of entry.

Whether a product is roll goods was originally decided by its category name, read from configuration. That holds for the shelves the shop started with but has two gaps: a category created today cannot be made roll goods without editing a configuration file, and a category is not always uniform — a boxed pre-cut kit sits among the film but is sold as one piece. Resolution is now a three-level fallback: a product's own setting, then its category's setting, then the category-name list. A product set in neither behaves exactly as it did before the feature existed.

7.5 Panel-based stock deduction

This is the central inventory feature of the system and the one the business identified as most important. A tint job is priced and recorded per panel: a line reads 'Front Windshield × 1'. Deducting the line's quantity would take one centimetre off the roll for work that consumes well over a metre. **The inventory would appear healthy while the roll ran out.**

The system therefore holds a cut size for each pairing of vehicle classification and panel, and on completion of a job order, service invoice or sale it posts an outbound transaction for the cut size rather than the quantity. The figure cannot live in the database — the pricing table is keyed on exactly the right columns but carries only a price — so it is held in a JSON file and edited through a grid on the Inventory module, with classifications down the side and panels across the top.

A pairing with no cut size **stops the document** rather than guessing. This is deliberate: a job that cannot be completed is noticed immediately and fixed in a minute, whereas a wrong number on the shelf is not noticed until a stock count, by which time the cause is weeks old. The blocking message names the pairing and links directly to the grid.

The grid shows only the pairings that film is actually priced for, read from the pricing table. The add-line forms build their *panel* list from the same source, so a pairing with no price can never reach a document and asking for its cut size would be busywork — 63 of 117 cells, in the shop's current catalogue.

7.6 Recording which shade was fitted

A tint film is sold in shades — Light, Medium, Superdark — and which one went onto a car matters to the job sheet, to the receipt, and to a warranty claim months later. ops.JobOrderDetail, ops.QuotationDetail and sales.ServiceInvoiceDetail have all carried a ShadeID column since the database was designed, and **nothing had ever written to one**. Every line in the system recorded the film and the variant but not the shade.

The add-line forms on all three documents now offer the shades belonging to the chosen tint variant, and the value is kept only where the shade genuinely belongs to that variant, so a stale selection left on the form after the variant changed cannot record a shade the film never came in.

Where the pickers get their options from, and why it mattered

The first implementation built both the variant and the shade picker from the pricing matrix, which was a conflation of two different questions. A film with no price row for the vehicle's classification therefore offered neither. BF Film has three variants and nine shades on file and not one pricing row, so selecting it produced N/A, N/A and a price of zero, and the job sheet could not say what went on the car. The server carried the same confusion and would have discarded the values even had the form posted them: a line kept its variant only when a pricing row was found.

The variant and the shade are a record of **what was fitted**. A shade has never been a price driver at all, and a variant only narrows which matrix row applies. They are therefore now read from cat.TintVariant and cat.Shade

and offered whether or not the product is priced per panel, while the panel and the price still come from the matrix, which is what the matrix is for. Corrected on job order, quotation and service invoice alike.

> ***What this does not do.*** *It does not separate stock. inv.InventoryTransaction records a ProductID and nothing finer, so every shade of a film draws on one balance and a deduction cannot say which shade left the shelf. This is a record of what was fitted, not a per-shade ledger; sections 7.11 and 7.12 cover how far the schema allows that to be addressed.*

7.7 Glass geometry: from the size of the glass to the length off the roll

The shop supplied its measured cut sizes as the *width and height of each piece of glass*, per panel per vehicle class. These are not the same as the length consumed, because film arrives on a roll of fixed width and the shop buys 30-inch (76.2 cm) stock. A windshield measuring 152.4 cm cannot be cut from a 76.2 cm roll in one piece; it is two strips laid side by side, consuming 304.8 cm of length.

The system therefore stores the measurements and computes the length. For a piece of width *w* and height *h* cut from a roll of width *R*, it evaluates both orientations and takes the cheaper:

$$\text{length} = \min\left(\operatorname{ceil}(w / R) \times h \;,\; \operatorname{ceil}(h / R) \times w \right)$$

Both orientations must be tried because which is cheaper is not obvious: a 119 × 51 cm rear screen is one strip of 119 cm but two strips of 51 cm. This is what a cutter does by eye. The roll width is a configuration setting, because every figure in the table moves when it changes — the same windshield is a single 152.4 cm strip off 60-inch film, for half the length.

Full Wrap is not stored as its own figure. It is computed as the sum of the four main panels — windshield, both rows and the rear screen — from the same measurements, so a full job and the panels that compose it cannot drift apart. The sunroof and quarter glass are excluded because they are priced as panels in their own right and including them would deduct them twice.

Independent corroboration of the model

The shop also supplied full-wrap figures measured directly: 304.8 cm for the small-sedan-to-MPV band, 342.9 cm for a big SUV and 543.56 cm for a van. Evaluated on 60-inch film, the computed four-panel sums are 347.98 cm for a big SUV (1.5% above the measured figure) and 528.32 cm for a van (2.8% below). That two independently supplied figures reconcile to within three per cent of a sum derived from a different set of measurements is strong evidence that both the geometric model and the decision to exclude sunroof and quarter glass are correct. The band figure of 304.8 cm sits about 14% below, consistent with its being a single rounded trade figure covering six vehicle classes.

7.8 Reversal integrity: the stock posting log

A document that is completed and later reverted must return *exactly* what it took. The first implementation recomputed the quantity from the current cut sizes at the time of reversal. This is wrong whenever a cut size is corrected in between: a job that took 1.20 m and is reverted after the figure is corrected to 1.35 m returns 1.35 m, silently creating 15 cm of film that does not exist. Worse, clearing a cut size entirely made reversal throw, stranding the film in a completed document with no way to release it.

A posting log was introduced that records what each document actually took. Reversal reads the log, and falls back to recomputation only when there is no record — which is the pre-existing behaviour and is correct for documents completed before the log existed. Verification confirmed that correcting a cut size from 1.20 m to 1.35 m mid-job and then reverting returns the roll to exactly 100.00 m, and that clearing every cut size still permits an exact reversal.

7.9 Quotation lifecycle

The quotation status column admits seven values at the database level: Draft, Sent, Accepted, Rejected, Converted, Cancelled and Expired. In practice the shop used two, and the intermediate states were a source of inconsistent records. The user interface was reduced to Draft and conversion to a job order.

Cancelled and Expired are preserved where they already exist on historic rows rather than being rewritten, and the displayed status is derived from whether a job order exists rather than from the stored text — so the label and the reality cannot disagree. The same predicate drives the report filter, so filter and label always agree.

7.10 Reporting and cashiering reconciliation

The reporting module aggregates sales, collections, inventory and service invoices, with PDF export. The headline figures on the dashboard and the reports overview originally summed the direct-sales table only, which made every service invoice invisible — a 42,000 peso tint job did not move the number at all. Both were corrected to include service invoices, counting only those with a status of Paid, which is the same rule the Cashiering screen applies. Because both now derive from a single shared calculation, **the reports and the cashiering screen reconcile by construction rather than by coincidence.**

Staff-facing collection reports withhold monetary totals. An early version printed ‘Total: 0.00’, which reads as ‘nothing was taken’ rather than ‘this figure is withheld’; it was changed to state explicitly that totals are shown to administrators only.

7.11 Counting stock shade by shade

The shop’s practical question — how much Superdark is left on the rack — cannot be answered by the running balance, for the reason given in 7.6: the transaction ledger records a product and nothing finer. Changing that is a schema change and was out of scope.

inv.InventoryCheckDetail, however, carries TintVariantID and ShadeID and had never used them. The physical count sheet therefore now lists a roll product once per shade — the first line carrying the product name and the system figure, the rest indented beneath it — and records which shade each figure was counted against. A film with no shades on file, and every piece good, keeps the single line it always had.

The variance remains a product-level figure, and this is the part that had to be right. Reconciling posts to the transaction ledger, which has no shade, and there is one system figure per product to compare against — so the shade counts are summed before being matched to it, both in the server-side calculation and in the live variance shown while the sheet is being filled in. One variance row per counted line would have compared each shade against the product’s entire stock and posted the adjustment once per shade. Verified against a database: three shades counted at 30, 25 and 40 metres against 100 metres on the books collapse to a single variance of five metres short, where the ungrouped arithmetic would have posted 205 metres.

7.12 Estimating what is left of each shade

Counting shade by shade gives a starting point. It does not, on its own, answer the question between counts. Two things in the system can be broken down by shade, however, and together they are enough for an estimate: a stock count records what was physically on the rack per shade, and every document line now records which shade was fitted. One gives the starting point, the other gives what has gone since.

left = counted at the last stock count − fitted on documents since

Inventory > Shade stock presents that per film, with the product’s real ledger figure beside it for comparison. The screen states plainly that the figure is an estimate. It is not a substitute for counting; it is what keeps the last count useful until the next one.

What it refuses to do is guess

A line carrying no shade cannot be charged to one, so it is reported as **unattributed** rather than dropped or spread evenly across the shades. A delivery is recorded against the film rather than a shade, so it cannot be added to one either. Where either has happened, the screen states that the shade figures and the ledger are not expected to agree, and which of the two is the reason — rather than displaying a gap and leaving the reader to puzzle over it. A shade that has never been counted has no starting point, and shows none.

Only documents that actually moved stock are counted, by the same rules the posting itself uses: a job order when it is Completed, and a service invoice when it is Paid *and* stands on its own. An invoice raised from a job

order never posts, because the job order already did, so counting it here would subtract the same film twice.

One approximation is made and is documented in the service itself. A document's date stands in for the moment its film left the shelf, because nothing records the latter: the ledger carries a date but no link to the document, and the posting log carries the link but no date. The approximation errs towards *under*-reporting what remains, which sends somebody to look at the rack, and it corrects itself at the next count. **An estimate that is wrong in the safe direction, and says so, is worth more than a precise figure that cannot be produced.**

Verified against a database: a job completed before the count is not subtracted again, one completed after takes its cut size, an unfinished job takes nothing, an invoice raised from a job order does not subtract twice, a standalone paid invoice does, and a shadeless line is surfaced rather than lost.

7.13 Recording a vehicle's condition at intake

A tint or PPF job puts the shop's hands on a customer's paintwork and glass for several hours. If a mark is disputed afterwards, the only thing that settles it is a record of the car's condition *before* work began, agreed at the time. The module existed from early in the project and recorded the right thing; what it did not do was make recording it practical.

Four faults, and what each cost

Fault	Consequence
Each panel was its own form, posting to its own action and redirecting back.	A seven-panel car was seven full page loads — typed on a phone, in a workshop. The create screen produced a header with no panels at all.
The condition was a free-text box.	"Good", "good" and "ok lang" were three different answers to one question. Nothing could be counted or compared.
The list showed six columns of names and dates.	A sheet nobody had started looked exactly like a finished one, and no screen anywhere said what a checklist had actually found.
The panel list came from the pricing catalogue, in alphabetical order.	"Full Wrap" and "Whole Vehicle" are prices covering several panels at once, so the windshield could be recorded twice; and the walk began at "2FWS" with the windshield third.

The walkaround is now a single form: the header and every panel post together and save once, on the create screen and on the details screen alike. The condition is a choice — Good, Old film / residue, Scratched, Chipped, Cracked, Stained, Not applicable — stored in the same text column as before, so **every condition typed by hand up to now reads back exactly as it was typed** and is offered back as its own option rather than being quietly replaced. Anything unrecognised is reported as *noted* rather than as damage: the shop wrote it, and the shop can mean it.

Panels are ordered front to back, the way a person walks a car, and the pricing bundles are off the sheet — though a bundle that already carries a recorded condition is still shown, because hiding a row is worse than offering one nobody wants. The list now reports what each car arrived in: issues found, how far the walkaround got, and which faults. One checklist per job order is enforced, with the job order dropping off the picker once it has one and a duplicate refused on post with a link to the sheet that exists. The typed walkaround survives that refusal, because whoever typed it walked the car to get it.

Two smaller corrections follow from thinking about when each field can be known. The client confirmation could previously be set only on the create form and never again — but the customer signs *after* seeing the walkaround, so it is now editable on the details screen. And a panel cleared back to blank now loses its row, so that 'never looked at' and 'looked at, said nothing' remain different answers.

Finally the record is issuable. A PDF carrying no prices, the panels in plain language, the line stating that anything not written down was not found on arrival, and the signature. Panels nobody inspected print as *Not inspected* rather than being omitted: **a gap the customer can see is a gap the shop cannot later fill in.** The job order screen gained the way in, since the job order is where the car arrives.

8 Presentation and the Design System

Correctness had been pursued screen by screen, and the presentation had drifted the same way. This section documents a late phase of work that treated the interface as a system rather than as a collection of pages, and the defect that exercise uncovered.

8.1 The measured problem

Ninety of the application’s hundred and five views carried their own embedded style block re-declaring the same handful of classes. The duplication was measured before anything was changed, because the scale of it is the argument:

Class	Times defined	Distinct definitions
.zx-card	84	23
.btn-zx	80	8
.zx-page-head	30	7

That is the reason no two screens quite matched. A card on one screen had 22px of padding and a 480px cap; the same card on another had 24px and 640px; and **nobody had ever decided which was right.** Fixing the look screen by screen would have meant making that decision ninety times, and would have produced a ninety-first variation.

8.2 The dashboard as the reference point

A single screen was redesigned first, to settle what the language should be. The old dashboard gave its best position — a dark banner across the top — to a sentence stating how many modules the signed-in role granted. That is a fact about permissions; it never changes between visits and nobody can act on it. Beneath it, the figures that *can* be acted on sat in identical grey cards, and below those, sixteen module tiles of equal weight each carrying an ‘Open’ link.

Three changes, in the order they matter. The top of the page became a greeting, the date, and the things people open the screen to start — New Sale, New Job Order, New Quotation, Cashiering — each gated on the same module claims as the tiles. The figures gained a *tone*: stock running out is red, work waiting is amber, takings and warranties are plain. The tone follows **the number rather than the kind of number**, so nothing-waiting does not look like a warning with a zero in it, and each figure gained a line of context because a bare 2 does not say whether 2 is bad. The module grid became a directory again rather than the main event.

The whole change is cosmetic apart from the stat tone, which is one optional field defaulting to Plain — so a statistic added later without thinking about it reads as information rather than as an alarm. Nothing outside the dashboard touched any of it, which meant the commit could be reverted on its own if the shop disliked it.

8.3 Consolidation

With the language settled, the shared page furniture — headings, cards, tables, buttons, row-end icon buttons, search boxes, back links, status pills and empty states — was defined once in site.css, and 2,238 lines of duplicate CSS came out of the views. Each screen kept whatever was genuinely its own.

Three of the variations turned out to mean something and were kept as *modifiers* rather than flattened away: *.padded* for a form card, *.narrow* to stop a short form stretching a metre wide, and *.danger* for the pink border on a delete confirmation, which is a warning and not drift. The five different form widths between 480 and 720 pixels collapsed to one, which was the point of the exercise. The authentication layout was excluded entirely: the login screen is the one page that should not look like a data grid.

Before any declaration was removed, an analysis was run of what removing it would *lose*. That caught three regressions before they shipped rather than after: stacked cards lost the gap between them, search boxes lost their clearance above a table, and page headings dropped to a gap that only looks right when a sub-paragraph follows. All three were fixed in the shared stylesheet first.

8.4 What consolidation exposed

The shared stylesheet packs two label/value pairs onto each row of a detail list on a wide screen, which turns a thirteen-field invoice from a thirteen-row list into a five-row block. Two pairs need 658 pixels. The *.narrow* modifier caps a card at 640. **The last value on every narrow card was therefore being clipped at the card's edge** — a date reading '02 Oct 2026, 9:15' with the AM cut off.

This affected sixteen screens, essentially every delete confirmation in the application, and was found only because a later feature rendered one of those screens and the result was looked at. The fix is to exclude narrow cards from the two-up grid, so they keep one pair per line, which is what a 640-pixel card has room for.

> ***On the cost of a sweeping change.*** *The defect was introduced by the consolidation commit itself and found two commits later. It is recorded here, and as defect 24, on the same footing as the inherited ones. A change that touches eighty-seven views at once buys consistency and takes on exactly this risk in exchange; the mitigation is that there is now one place to fix it rather than sixteen.*

9 Verification and Quality Assurance

9.1 Approach

The system has no automated test suite in the repository. Verification was instead carried out by building purpose-made harnesses for each class of risk and running them before the work was committed. The reasoning is that **the failures that matter in this domain are silent** — a wrong stock figure, a centavo that does not reconcile, a receipt that prints the wrong unit, a value clipped at the edge of a card — and these are not caught by exercising the user interface by hand. The techniques used are set out below, and this is acknowledged as a limitation in section 14.

9.2 Techniques

Technique	Purpose and notable findings
Backup file analysis	The exact text of the database's CHECK constraints, foreign keys and indexes was recovered by extracting UTF-16 strings from the SQL Server backup file. This established the authoritative rules that application validation was then derived from, rather than guessing at them.
In-memory EF Core harness	A SQLite-backed subclass of the application's own DbContext, used to exercise inventory posting, reversal, shade-stock derivation and resolution logic against real EF Core behaviour. Required three adaptations: suppressing computed-column SQL whose bracket syntax SQLite rejects, decimal-to-double converters because SQLite cannot sum decimals, and disabling foreign-key enforcement.
In-process rendering harness	The real application hosted in process with only its database and its sign-in replaced, so that pages are produced by the real controllers, the real Razor views and the real pipeline rather than by a reconstruction. Introduced after an earlier hand-built preview gave a false result. This is how the checklist screens of 7.13 were confirmed, and how the clipping defect of 8.4 was found.
Live ASP.NET harness	A real ASP.NET Core application stood up to settle framework-behaviour questions empirically rather than from memory — that an empty form value binds to null for a nullable boolean, that Razor omits a false boolean attribute, and that the form tag helper suppresses its own antiforgery token when one is already present.
Browser automation	Playwright with Chromium, driving the rendered pages and in several cases the application's actual script files, used to compare client-side calculations against the server across the full range of inputs, to measure layout overflow at desktop and phone widths, and to settle questions about event behaviour empirically. This is how the VAT rounding disagreement of 7.1 was found and proved fixed, and how the jQuery event defect (19) was diagnosed rather than guessed at.
PDF text extraction	Generated receipts and checklists were parsed back to text and asserted on, to confirm that line items, units, measured lengths, panel conditions and included-at-no-charge services print as intended — and, when a shared helper was changed, that the four documents already using it still read as they did before.
Arithmetic cross-checks	Independently supplied figures reconciled against computed ones, as described in 7.7.
Review sweeps	Dedicated defect-hunting passes over completed work, which together found fifteen defects that normal development had not surfaced. These are listed in section 10.

10 Defect Log

The following defects were found and fixed during development. They are included because the nature of the failures, and how each was detected, is more informative about the system's risk profile than a list of features would be. **Every one of them produced plausible-looking output.**

#	Defect	Cause and consequence	Resolution
1	**Schema change broke the live application**	A column added to the product table did not exist in the running database; the application failed with an invalid-column error on every affected screen.	Reverted in full. The schema was treated as fixed thereafter, and the side-file pattern of 4.4 was adopted.
2	**Short payments accepted on non-cash modes**	Exact-amount clamping ran before the short-payment guard, making the guard unreachable. A 2,000 peso payment against a 2,500 peso invoice recorded as paid in full.	Clamp only upward, so a genuine shortfall still reaches the guard.
3	**Browser and server disagreed on VAT**	JavaScript double arithmetic rounds 6.30 / 1.12 differently from decimal arithmetic. Approximately 2% of totals displayed a centavo adrift of the stored figure.	Rewrote the client calculation in integer centavos. Zero disagreements across 54,055 totals.
4	**Stock reversal returned the wrong quantity**	Reversal recomputed from the current cut size, so a figure corrected between completion and reversal silently created or destroyed film. Clearing a cut size made reversal throw.	Introduced a posting log recording what each document took, with recomputation as fallback.

#	Defect	Cause and consequence	Resolution
5	**Sale completion would have failed on every sale**	Change was passed through TempData as a decimal, which the default serializer cannot serialise and throws on.	Passed as a formatted string. Caught on the bench before reaching the user.
6	**Reports and dashboard understated takings**	Headline figures summed direct sales only, so every service invoice was invisible.	Included service invoices with status Paid, matching the cashiering rule, via a shared calculation.
7	**Proof files downloaded instead of opening**	Supplying a file name to the file result sets a Content-Disposition of attachment, scattering customers' bank receipts into the Downloads folder.	Dropped the file name so the browser displays the file inline.
8	**Valid uploads could be rejected as forgeries**	The magic-byte check assumed a single read returns all twelve bytes; a short read could reject a valid file and accuse the user of renaming it.	Read in a loop until the buffer is filled or the stream ends.
9	**Job order receipts silently broken**	A text replacement intended for the PDF action matched an earlier occurrence, duplicating an include and breaking every job-order receipt.	Found in review and corrected; the same class of error occurred twice and was then guarded against.
10	**NULL materialisation crashed every customer list**	Columns declared non-nullable in the model held NULL in the live database.	Model hardened against pre-existing NULL data in three places.
11	**Payment mode seeding would have created duplicates**	The default offered list still named the legacy names that the rename map renames away, so without a configuration section it would rename rows then re-insert the old names.	Default list corrected to the post-rename names.
12	**Staff collections report read as zero takings**	Withheld totals printed as 0.00, which reads as 'nothing was collected'.	Replaced with an explicit statement that totals are administrator-only.
13	**Plate validation rejected valid plates**	The pattern did not admit motorcycle-style plates, including one already present in the live data.	Widened to up to three letters and four digits in either order, then narrowed again to car formats once the shop confirmed it does not take motorcycle work.
14	**Add Line Item silently did nothing**	A service invoice line could be submitted with no effect and no error.	Corrected, and the add-line forms aligned across quotation, job order, invoice and sale.
15	**Cut sizes were invented, not measured**	The initial table was estimates scaled from one assumed sedan windshield, and treated glass dimensions as roll length.	Replaced with the shop's measured glass sheet and the strip geometry of 7.7.
16	**A rule change left two of its three places behind**	Widening the plate rule changed the regular expression on the model only. The browser pattern attribute went on refusing what the server now accepted, making the widened rule unreachable through the form; and the normaliser still assumed the letters came first, silently storing "2956 XB" as "295 6XB".	All three brought into step and verified together against eleven plate forms. The normaliser now splits on the real letter/digit boundary rather than a fixed position.
17	**Requiring a plate broke reading the plateless ones**	[Required] is not only a validation attribute — EF Core reads it as a mapping rule and marks the column non-nullable, so the materializer stopped checking for DBNull and every vehicle already on file without a plate threw SqlNullValueException on read. This took out the vehicle list and every picker that joins a vehicle in.	IsRequired(false) in the model configuration states what is true of the column; the form still enforces the requirement at the ModelState level. The context already guarded Customer.FullName the same way, from the identical failure earlier in the project.

#	Defect	Cause and consequence	Resolution
18	**Editing an invoice bypassed exact settlement**	Exact-amount settlement was applied where payment is taken but not where an invoice is edited, so the rule could be walked around afterwards: 50,000 typed against a 42,000 card invoice stored 8,000 of change, and the reprinted receipt showed it.	Edit now sets the tendered amount to the total for every mode but cash, and the form fills the field and locks it so the figure on screen is the figure that saves.
19	**The unit picker never fired, system-wide**	The searchable dropdowns committed a choice with jQuery's trigger("change"), which wakes only handlers bound through jQuery; the unit script listens with addEventListener and never heard it. Picking film left the unit list on pieces and the stock warning blank on every add-line form in the application.	The dropdown now dispatches a real DOM event, which jQuery also catches because it binds natively underneath. Confirmed in a browser before and after rather than reasoned about.
20	**Line descriptions carried the stock figure**	The description was auto-filled from the option's label, which includes "(98.98 m in stock)" to make the picker readable — so lines saved, and receipts printed, as "Black Series (98.98 m in stock)".	The clean name now travels on a data attribute; the label is only a fallback.
21	**A film with no prices offered no variant and no shade**	Both pickers were built from the pricing matrix, conflating what a line costs with what was fitted. BF Film has three variants and nine shades on file and no pricing rows, so selecting it gave N/A, N/A and a price of zero. The server would have discarded the values even had the form posted them.	Variant and shade now read from cat.TintVariant and cat.Shade, offered whether or not the product is priced per panel. The panel and the price still come from the matrix.
22	**Rounding a copied price tier cut a live price**	Rounding to the nearest 100 was applied to the tier copied verbatim from Black Series as well as to the marked-up tiers, turning an existing 2,250 into 2,200 on two panels — a silent price cut.	Only the marked-up tiers are rounded, and half away from zero so a .5 never drifts down. Caught by a verification pass before the script was run.
23	**Railguard prices existed only as prose**	Every Topflip tonneau listing offers a railguard with a price, but those prices lived inside the covers' description text. A fitter could read that a railguard costs 1,800 and still had no way to put one on a job order, so it went on as a typed line with a typed price, or not at all.	Added as products in their own right, filed with the covers they belong to, inside the existing idempotent seed script.
24	**Detail values clipped on sixteen screens**	The two-pairs-per-row grid introduced by the stylesheet consolidation needs 658px; the narrow-card modifier caps a card at 640. The last value on every narrow card was cut off at the card edge, across essentially every delete confirmation in the application.	Narrow cards are excluded from the two-up grid and keep one pair per line. Found by rendering one of the affected screens and looking at the result.
25	**The checklist offered pricing bundles as panels**	The walkaround built its panel list from the pricing catalogue, which carries "Full Wrap" and "Whole Vehicle" — prices covering several panels at once. The windshield could be recorded twice, once on its own and once inside a bundle, and the panels were in alphabetical order rather than the order someone walks a car.	Bundles are off the sheet, except where one already carries a recorded condition, and the panels run front to back.

Defects 16, 17, 20, 22, 24 and 25 were introduced by changes made during this project rather than inherited, and 17 reproduced a failure the codebase had already been bitten by and guarded against elsewhere. They are listed on the same footing as the rest deliberately: **a defect log that records only inherited faults describes the codebase it was given, not the work done on it.**

11 Data Management and Deployment

11.1 What must travel together

Git carries the code. It does not carry the SQL Server database, which lives on each machine. A teammate who pulls the branch therefore gets the application and an empty or stale database behind it. Three artefacts must be refreshed *together* for every installation to behave identically:

Artefact	How it travels	Location
The database	A SQL Server backup, committed	Database/ZEmpireUpdated.bak
Cut sizes	A JSON file, committed	App_Data/cut-sizes.json
Product measures	A JSON file, committed	App_Data/product-measures.json
Settings	Committed already	appsettings.json

They must be refreshed together because both JSON files are keyed by the database's own identifiers — cut sizes by classification and panel, measures by category and product — so a file taken from one database points at the wrong panels and the wrong products in another.

11.2 Backup and restore

The repository carries a documented runbook covering both directions: taking the backup in SQL Server Management Studio, by the Object Explorer click-path and by the equivalent T-SQL, and restoring it with the logical file names and the WITH MOVE clause that a different instance requires. It also documents the two failure modes encountered in practice — operating system error 3, a path that does not exist, and error 5, where the SQL Server *service account* rather than the signed-in user lacks permission to write into a folder under a user profile — and the workaround of backing up to the instance's own backup folder and copying the file in.

A warning is recorded about repository size: the backup is approximately 16 MB and Git retains every version permanently, so ten refreshes add 160 MB that cannot be reclaimed by deleting the file. Git LFS is identified as the remedy if refreshes become routine. The text seed scripts beside the backup are the diffable alternative, and rebuild the catalogue from nothing, but do not carry transactions.

11.3 What deliberately does not travel

Proof-of-payment and warranty-claim files are excluded, for the privacy reasons set out in 5.4. The stock posting log is excluded because it changes on every completion and would conflict on nearly every commit; its absence degrades gracefully. The connection string differs between machines — a default instance against SQL Express — and the runbook recommends marking appsettings.json skip-worktree locally so the change is not repeatedly committed.

11.4 Catalogue seeding

Five SQL seed scripts carry the catalogue: the 2026 price list, the coating and window tint per-panel pricing, the Profilm and PPF pricing, the BF Film per-panel pricing, and a vehicle model guide. All are idempotent in the strong sense — a product is created only when neither its new name nor any name it is already known by exists, so a list that renames an existing product attaches prices to the existing row rather than creating a second one. The 2026 price list alone carries 193 INSERT statements behind 193 guards, none unguarded. Nothing is updated or deleted, so re-running a script never overwrites a price edited in the application.

Each script states its mapping assumptions in a header comment for review. One of them states something stronger. The BF Film script carries the only figures in the catalogue that nobody at the shop supplied: its Stone tier copies the existing Black Series prices exactly, while the Pro and Supreme tiers are those plus 35% and 75%. Those two percentages were asked for and not given, so **they are named as a stated starting point in the script header, in the repository README and in this report, rather than being allowed to pass as the shop's prices.** The shop's own variant tiering was considered as a source and rejected: it runs between 1.8× and 2.3× across tiers with a 0.38× to 2.25× spread across panels, which would have produced noise rather than

a price list.

Three further seeders run in C# at application startup rather than as SQL: Identity roles and passwords, payment modes, and tint shades. These require no manual step, which matters because the shop’s own staff pull the branch and run the application without a database administrator.

12 Development Chronology

Period	Phase	Principal work
20–28 Aug	Foundation	Project scaffolding; controllers and service interfaces; Entity Framework rebound to the existing ZEmpire database as database-first; login, role-based access control and the Admin/Staff dashboards carrying the business's existing visual identity.
1–7 Sep	Core CRUD	Customer, Vehicle and Vehicle Classification screens; Product views; Employees and the Roles and Permissions screen; Job Types, Job Orders, Pricing, Price History and Inventory modules; quotation and service invoice controllers.
11–15 Sep	Catalogue and usability	Coating, window tint, Profilm and PPF pricing seeded; searchable selects across large dropdowns; interactive dashboard; job order to service invoice connection; the Add Line Item form connected to the real pricing matrix; employee login accounts and role assignment; password change and reset; responsive layout for narrow screens.
20–22 Sep	Hardening and Cashiering	A series of defensive fixes against live data — NULL materialisation, number generation, duplicate plates, administrator lockout; the Cashiering module built over Sales and Service Invoices with no database change; database CHECK constraints surfaced as form validation; warranty connected to the lines it references.
23–25 Sep	Measurement	Film and PPF measured by length with centimetre/inch/metre conversion; stock movement on job and invoice completion; stock counts; roll-goods categories moved into configuration; products grouped by category with duplicate shelves folded together; the 2026 price list seeded idempotently.
27–28 Sep	Payments and cut sizes	Pending invoice status, payment with change and the 12% VAT breakdown; payment on the direct sale screen; four payment modes with proof of payment, deletable and extended to warranty claims; itemised receipts; quotation reduced to two states; panel-based stock deduction built and the Cut Sizes grid added; the Collections report linked to Cashiering.
29–30 Sep	Payment policy and reporting	Payment modes narrowed to four with retired modes pruned where unreferenced; exact-amount settlement; card receipts required; service invoices included in the headline figures; repository data made shareable; the odometer removed, the plate rule widened and tint shades seeded; the backup runbook documented.
1 Oct	Catalogue administration	A Product Categories screen with its own measurement setting; per-product unit of measure overriding the category; and the estimated cut sizes replaced with the shop's measured glass sheet and the roll geometry derived from it.
1 Oct	Shop rules and a defect sweep	Plate numbers narrowed to car formats and made mandatory, with the regression that caused fixed; catalogue prices restricted to administrators; the shade recorded on quotation, job order and invoice lines; exact settlement enforced when an invoice is edited; the stock count sheet broken down by shade; and three faults on the add-line forms corrected, one of which had silently disabled the unit picker across the whole application.
2 Oct	Documentation	The first edition of this development report compiled from the repository.
7 Oct	Shade stock and the catalogue audit	The per-shade stock estimate derived from the last count and the documents since, with unattributed lines and deliveries surfaced rather than spread; the shop's full price list audited line by line against the system, which found railguards priced only inside prose; the variant and shade pickers moved off the pricing matrix; and BF Film priced per panel across three variants and eight vehicle classes.
10 Oct	The interface as a system	The dashboard redesigned around what people come to it to do, with figures carrying a tone that follows the number; that language then applied across every screen from one stylesheet, removing 2,238 lines of duplicated CSS; and the vehicle checklist rebuilt as a single-form walkaround with a fixed condition vocabulary, a findings summary and a signed PDF for the customer.

13 Project Metrics

Measure	Value
Development period	20 August 2026 – 10 October 2026 (7.5 weeks)
Commits on the Programmer branch	188
Lines of C# and Razor	29,198
Controllers	24
Service classes	15, behind 13 interfaces
Razor views and partials	108
C# source files	91
Mapped database entities	38, across 7 schemas
Modules in the permission matrix	16
Shared stylesheet	746 lines, replacing 2,238 lines across 90 views
Application scripts (excluding libraries)	3
SQL seed scripts	5, all idempotent
Startup seeders	3 (Identity, payment modes, tint shades)
Documented defects found and fixed	25 (section 10)

14 Limitations and Future Work

14.1 Known limitations

- **No automated test suite in the repository.** Verification was carried out with purpose-made harnesses run before committing, as described in section 9, but these were not retained as a regression suite. This is the most significant gap: a future change can reintroduce a fixed defect without anything failing.
- **Tint variants and shades share a stock pool.** The inventory transaction table records a product but not a variant or shade, so all shades of a film draw on one balance. The count sheet records the shade mix (7.11) and the Shade Stock screen estimates what is left from it (7.12), but that remains an estimate between counts. A live per-shade balance needs a schema change.
- **The shade estimate approximates when film left the shelf.** A document’s date stands in for the moment of deduction, because nothing records the latter. The error is in the safe direction and self-correcting at the next count, but it is an approximation.
- **Side-file state is tied to one database.** The JSON files are keyed by database identifiers and are meaningless against a different instance. This is documented, but it is a coupling that a schema column would not have.
- **Partial payment is not representable.** The database constraint forbids a stored change amount below zero, so an invoice is either settled in full or not at all. This reflects current shop practice rather than a general billing model.
- **Cut sizes are per classification, not per model.** Two vehicles in the same class with materially different glass take the same figure.
- **Hatchback cut sizes are inferred.** The shop’s measured sheet does not cover the hatchback class, which is currently given the small-sedan figures and flagged for measurement.
- **Most tint variants have no shades on file.** Five of thirteen carry them — the BF Film and Profilm lines supplied by the shop. Lines on the other eight record no shade, and the forms say so explicitly rather than reading as though none applied.

- **Three BF Film tiers carry derived prices.** The Pro and Supreme markups of 35% and 75% were asked for and not supplied. They are stated as a starting point in three places (11.4) and need the shop’s confirmation before they can be called the shop’s prices.
- **Default prices are not journaled.** Restricting them to administrators (5.3) controls who may change one, but there is still no record of a change having been made, because no table exists to hold it. The pricing matrix is journaled; the default price is not.
- **The vehicle checklist records condition but not photographs.** A written note is weaker evidence than an image, and the proof-of-payment mechanism already demonstrates that the application can store files safely.

14.2 Future work

- Retain the verification harnesses as an automated regression suite, beginning with the inventory posting, the VAT calculations and the checklist findings arithmetic. The in-process rendering harness of section 9.2 is the natural foundation, because it already drives the real application.
- Attach photographs to a vehicle checklist, reusing the existing evidence-file mechanism.
- A pricing grid mirroring the cut-size grid, so prices can be maintained in one screen rather than row by row.
- Roll cost tracking, to report margin per panel rather than revenue alone.
- CSV export across the reporting module, and PDF export for the service invoices report.
- Catalogue entries for installation-only services, for customers who supply their own product.
- Per-model cut sizes where a class proves too coarse.
- Shades for the eight tint variants that carry none, so every film line can record one.
- Confirmation or correction of the derived BF Film markups.
- A journal for default-price changes, matching what the pricing matrix already has. This needs somewhere to put it, so it is the clearest candidate if a schema change is ever sanctioned — as is a variant and shade on the inventory transaction, which would give a live per-shade balance and retire two of the limitations above.

A Appendix A — Complete Commit History

All 188 commits on the Programmer branch, oldest first, as recorded by Git.

#	Date	Author	Subject
1	2026-08-20	Sean	Add .gitattributes and .gitignore.
2	2026-08-20	Sean	Add project files.
3	2026-08-20	Sean	Added Controllers
4	2026-08-21	Sean	Fixed Services and Interfaces. Fixed Program.cs minimal changes
5	2026-08-28	BENJO LOPEZ	Rebind EF Core to the ZEmpire database (database-first)
6	2026-08-28	BENJO LOPEZ	Added: Account Login RBAC, Admin and Staff Dashboard Tiles. The Theme Layout Get in Wixsite of Z-Empire Website.
7	2026-09-01	SeanEsperancilla	Merge pull request #2 from SeanEsperancilla/Programmer
8	2026-09-03	BENJO LOPEZ	Modify: Admin and Staff Dashboard (Catergory and Icons)
9	2026-09-03	BENJO LOPEZ	Added: Customer Full CRUD and View Design.
10	2026-09-03	BENJO LOPEZ	Added: Vehicle Full CRUD and View Design (shared CRM styles, Customer/Vehicle links, dashboard tile)
11	2026-09-03	Sean	AppSettings
12	2026-09-03	Sean	Merge branch 'master' into Programmer
13	2026-09-03	Claude	Add CRUD views for Product, Vehicle, and VehicleClassification
14	2026-09-03	SeanEsperancilla	Merge pull request #3 from SeanEsperancilla/claude/home-index-chtml-review-yf1yp6
15	2026-09-03	Claude	Add Customer CRUD views and block deletion while vehicles are registered
16	2026-09-03	Claude	Add Employees module and a Roles & Permissions screen
17	2026-09-03	SeanEsperancilla	Merge pull request #4 from SeanEsperancilla
18	2026-09-04	Sean	Add Job Types, Job Orders, Pricing, Price History, and Inventory modules
19	2026-09-04	Sean	Add Job Types, Job Orders, Pricing, Price History, and Inventory modules
20	2026-09-04	Sean	Revised and Added Controllers, Fixed Quotation Service and Interface. Added Views for Controllers
21	2026-09-04	Sean	Added Updated Database
22	2026-09-05	Sean	Revised: QuotationController, added user for Create and Edit
23	2026-09-05	Sean	Fixed Bugs at ServiceInvoiceController
24	2026-09-05	Sean	Revised UI design, Need to fix User Management's UI
25	2026-09-07	Sean	Employees View (Details and Index)
26	2026-09-11	Claude	Validate vehicle plate number format (3 letters + 4 digits)
27	2026-09-11	Claude	Add warranty auto-expire and Claim Warranty action
28	2026-09-11	Claude	Add search bars to Quotation, Job Order, and Sales lists
29	2026-09-11	Claude	Add Post/lock action for Quotation and Job Order
30	2026-09-11	Claude	Add PDF generation for Sales, Quotation, and Job Order reports
31	2026-09-11	Claude	Improve Customer<->Vehicle display, fix swapped Details/Delete views
32	2026-09-11	Claude	Add per-record PDF generation for Quotation, Sales, Service Invoice, Job Order

#	Date	Author	Subject
33	2026-09-11	Claude	Cascade Vehicle dropdown from Customer selection
34	2026-09-11	Claude	Full UI design audit: fix remaining plain-button/unstyled views
35	2026-09-11	Claude	Fix Employee UI buttons and add 3-tier Inventory stock status
36	2026-09-11	Claude	Direct new customers to Add Vehicle; validate Philippine plate formats
37	2026-09-11	Claude	Replace native confirm() on line-item Remove with a styled modal
38	2026-09-11	Claude	Add Coating and Window Tint per-panel pricelists as seed SQL
39	2026-09-11	Claude	Add product search bar to the Pricing list
40	2026-09-11	Claude	Add Profilm Nano Ceramic Tint and Print Protection Film seed SQL
41	2026-09-11	Claude	Search Pricing by tint variant, vehicle classification, and panel too
42	2026-09-13	Claude	Make large dropdowns searchable instead of plain <select> lists
43	2026-09-13	Claude	Make the Home dashboard interactive: quick stats + module search
44	2026-09-13	Claude	Searchable selects: only show matches once typing, no browse-all dropdown
45	2026-09-13	Sean	Home Index.chtml
46	2026-09-13	Sean	Merge branch 'Programmer_Two' of https://github.com/SeanEsperancilla/ZEmpireAutoAccessories into Programmer_Two
47	2026-09-13	Claude	Show a Job Order's originating Quotation in the Job Orders listing
48	2026-09-13	Claude	Auto-select a customer's only vehicle; label CustomerID/VehicleID properly
49	2026-09-13	Claude	Auto-fill Price on the Add Line Item form when a product/service is picked
50	2026-09-13	Claude	Fix build: SelectListItem has no Attributes property
51	2026-09-13	Claude	Connect Add Line Item to the real Pricing matrix, not just flat prices
52	2026-09-13	Claude	Replace Convert confirm() with a modal; show Tint Variant/Panel on lines
53	2026-09-13	Claude	Remove Default Price column from Products list
54	2026-09-13	Claude	Connect Job Order to Service Invoice
55	2026-09-13	Claude	Cascade Job Order dropdown from Customer on Service Invoice forms
56	2026-09-13	Claude	Fix invoice total not saving when copying job order line items
57	2026-09-13	Claude	Fix CK_ServiceInvoice_Math violation on every ServiceInvoice save
58	2026-09-13	Claude	Fix CK_ServiceInvoice_Change violation - keep AmountPaid >= TotalAmount
59	2026-09-13	Claude	Add self-service Change Password and admin Reset Password
60	2026-09-13	Claude	Wire up the Dashboard tile on the module grid
61	2026-09-13	Claude	Give the Dashboard tile actual content instead of a bare link
62	2026-09-14	Claude	Fix crash on New Warranty page (null model)
63	2026-09-14	Claude	Auto-reactivate an Expired warranty when its end date moves forward
64	2026-09-14	Claude	Fix remove-line button doing nothing on the last row in New Sale
65	2026-09-14	Claude	Auto-price matrix-priced products (e.g. tint) on New Sale
66	2026-09-14	Claude	Surface a visible warning if the pricing matrix AJAX call fails
67	2026-09-14	Claude	Add search bar to Warranty list
68	2026-09-14	Claude	Hide Today's Sales peso figure from Staff on the Dashboard

#	Date	Author	Subject
69	2026-09-14	Claude	Remove the Manager role - Admin and Staff only
70	2026-09-14	Claude	Add show/hide password toggles to Change/Reset Password forms
71	2026-09-14	Claude	Fix invisible hamburger menu icon on mobile nav
72	2026-09-14	Claude	Make the UI properly responsive on narrow screens
73	2026-09-14	Claude	Allow changing an employee's login username on Edit
74	2026-09-15	Claude	Add Role assignment to Employee Create/Edit
75	2026-09-15	Claude	Create a real login account on New Employee instead of requiring one to already exist
76	2026-09-15	Claude	Reject duplicate EmployeeNumber on Employee Create/Edit
77	2026-09-20	Claude	Enforce required customer phone, unique plate numbers, auto quotation validity
78	2026-09-20	Claude	Guard Admin against locking itself out of the module permission matrix
79	2026-09-20	Claude	Fix SqlNullValueException when a product has no inventory transactions yet
80	2026-09-20	Claude	Harden number generation and invoice-number errors against SQL failures
81	2026-09-20	Claude	Fix SqlNullValueException on every list that includes a Customer
82	2026-09-20	Claude	Harden Customer.FullName the same way against pre-existing NULL data
83	2026-09-20	Claude	Remove the Posted status and its edit/delete lock from Quotation and Job Order
84	2026-09-20	Claude	Remove the self-referential Dashboard tile from the dashboard grid
85	2026-09-20	Sean	Remove Dashboard Tile
86	2026-09-22	Claude	Fix the same NULL-materialization crash across Reports and user records
87	2026-09-22	Sean	Sales (Add Items)
88	2026-09-22	Sean	Merge branch 'Programmer' of https://github.com/SeanEsperancilla/ZEmpireAutoAccessories into Programmer
89	2026-09-22	BENJO LOPEZ	Modify: Sales Create View.
90	2026-09-22	Claude	Merge branch 'Programmer_Two' into Programmer
91	2026-09-22	Sean	Appsettings
92	2026-09-22	Claude	Align app validation with two DB constraints found in the audit
93	2026-09-22	Claude	Add Cashiering module over Sales and Service Invoices
94	2026-09-22	Claude	Show Cashiering on the dashboard under Sales & Billing
95	2026-09-22	Sean	Views(Index chtml)
96	2026-09-22	Sean	Edited Views INDEX from Home
97	2026-09-22	Claude	Open Cashiering with no database change
98	2026-09-22	Claude	Merge remote-tracking branch 'origin/Programmer' into Programmer
99	2026-09-22	Claude	Resolve the conflict markers committed into the dashboard view
100	2026-09-22	Claude	Narrow Pricing's Tint Variant to the chosen product, and auto-fill singles
101	2026-09-22	Claude	Require customer contact numbers to be 11 digits starting 09
102	2026-09-22	Claude	Replace the type-to-search selects with plain dropdowns
103	2026-09-22	Claude	Make Customer and Product searchable again, everything else a dropdown
104	2026-09-22	Claude	Apply searchable to the last two Customer/Product pickers

#	Date	Author	Subject
105	2026-09-22	Claude	Make every select searchable, with an opt-out
106	2026-09-22	Claude	Restrict the Cashiering totals to Admin
107	2026-09-22	Claude	Stop listing Cashiering twice in the Modules menu
108	2026-09-22	Claude	Stop the dropdown being clipped, and widen the line-item table
109	2026-09-22	Claude	Align two relationship gaps found re-auditing the model against the DB
110	2026-09-22	Claude	Connect Warranty to the sale and invoice lines it already references
111	2026-09-22	Claude	Make the warranty hand-off from a sale or invoice readable
112	2026-09-22	Claude	Make the money and date rules the database enforces visible on the forms
113	2026-09-22	Claude	Stop the invoice silently overwriting the amount tendered
114	2026-09-22	Claude	Fix Add Line Item silently doing nothing on a service invoice
115	2026-09-22	Claude	Sales: stock warnings, wider layout, phone layout, receipt on completion
116	2026-09-23	Claude	Point the connection string at the default instance
117	2026-09-23	Sean	Appsettings
118	2026-09-23	Sean	Merge branch 'Programmer' of https://github.com/SeanEsperancilla/ZEmpireAutoAccessories into Programmer
119	2026-09-23	Sean	Appsettings
120	2026-09-23	Claude	Revert the connection string to the SQLEXPRESS instance
121	2026-09-23	Claude	Merge origin/Programmer, keeping Server=. from the remote
122	2026-09-23	Claude	Set product prices from the Product module
123	2026-09-23	Claude	Use the full screen width across the CRUD screens
124	2026-09-23	Claude	Move stock on job/invoice completion, and add stock counts
125	2026-09-23	Claude	Validate the three money fields the database guards but the app did not
126	2026-09-23	Claude	Make the inventory screens quicker to work
127	2026-09-23	Claude	Measure film and PPF by length, converting between cm, inches and metres
128	2026-09-23	Claude	Sell film by the metre on a direct sale
129	2026-09-23	Claude	Say on the Product screens how a product is measured
130	2026-09-23	Claude	Align the Add Line Item forms and connect them to stock
131	2026-09-25	Claude	Organise the Inventory list around units and categories
132	2026-09-25	Claude	Treat the Tint and Paint Protection categories as roll goods
133	2026-09-25	Claude	Choose per product whether it is measured or counted
134	2026-09-25	Claude	Revert "Choose per product whether it is measured or counted"
135	2026-09-25	Claude	Keep the roll-goods list in configuration instead of the database
136	2026-09-25	Claude	Seed the 2026 pricelist without duplicating what is already there
137	2026-09-25	Claude	Group the Pricing list by product, variant and panel
138	2026-09-27	Claude	Add scripts to merge the duplicate categories and clear dead products
139	2026-09-27	Claude	Show one heading per shelf instead of one per category name
140	2026-09-27	Claude	Group the Products list by category

#	Date	Author	Subject
141	2026-09-27	Claude	Use a peso sign on the Set price button, not a dollar
142	2026-09-27	Claude	Fix the line-item unit list and keep it in step with the server
143	2026-09-27	Claude	Start invoices unpaid, take payment with change, and show the 12% VAT
144	2026-09-27	Claude	Add a reprint button to the cashiering and service invoice lists
145	2026-09-27	Claude	Take payment on the New Sale screen, with change and the 12% VAT
146	2026-09-27	Claude	Offer the four payment modes, and attach proof of an online payment
147	2026-09-27	Claude	Spell out every line on the receipt, and say what was not charged for
148	2026-09-27	Claude	Let a wrongly attached proof be removed, and evidence a warranty claim
149	2026-09-27	Claude	Reduce a quotation to two states: Draft, or converted to a job order
150	2026-09-27	Claude	Say "Transactions" on the cashiering breakdowns instead of "Txns"
151	2026-09-27	Claude	Deduct film by the cut size of the panel, not by the line's quantity
152	2026-09-27	Claude	Put a way out on the missing-cut-size banner
153	2026-09-27	Claude	Put Cut sizes on the Inventory header
154	2026-09-27	Claude	Fill the cut sizes from a starting table, and say what unit a box is in
155	2026-09-28	Claude	Add whole-vehicle and all-glass cut sizes, from what those panels price
156	2026-09-28	Claude	Say what a panel line takes off the roll, instead of "pc"
157	2026-09-28	Claude	Print the film a panel line used on the receipt, not "pc"
158	2026-09-28	Claude	Fix nine defects found reviewing this session's work
159	2026-09-28	Claude	Add a Collections report, and link it to Cashiering both ways
160	2026-09-29	Claude	Make the shared data travel with the branch
161	2026-09-29	Claude	Offer Bank Transfers, Credit Cards, GCash and Cash, and nothing else
162	2026-09-29	Claude	Delete retired payment modes, where nothing was taken on them
163	2026-09-29	Claude	Settle an online payment at the exact amount, with nothing to type
164	2026-09-29	Claude	Charge cards the exact amount too, and stop conflating that with proof
165	2026-09-29	Claude	Ask for the slip on a card payment too
166	2026-09-29	Claude	Count service invoices in the headline figures, not product sales alone
167	2026-09-29	Claude	Fix six defects found reviewing the payment and reporting work
168	2026-09-29	Claude	Show the cut-size grid only the pairings film is actually priced for
169	2026-09-30	Claude	Drop the odometer, widen the plate rule, and seed the tint shades
170	2026-09-30	Sean	cut-sizes.json
171	2026-09-30	Claude	Document the SSMS backup click-path and its two failure modes
172	2026-10-01	Claude	Add a Categories screen, and let a product set how it is measured
173	2026-10-01	Claude	Replace the estimated cut sizes with the shop's measured glass sheet
174	2026-10-01	Claude	Make the plate rule agree with itself across the form and the normaliser
175	2026-10-01	Claude	Four shop rules: car plates only, admin prices, shade on a line, exact edits
176	2026-10-01	Claude	Require a plate number on a vehicle
177	2026-10-01	Claude	Fix SqlNullValueException on every screen that reads a vehicle

#	Date	Author	Subject
178	2026-10-01	Claude	Say when a tint variant has no shades on file
179	2026-10-01	Claude	Count film shade by shade on the stock count sheet
180	2026-10-01	Claude	Fix three faults on the Add Line Item forms
181	2026-10-02	Claude	Bring the development report up to date
182	2026-10-07	Claude	Work out what is left of each shade, from the count and the job orders
183	2026-10-07	Claude	Make railguards sellable, not just mentioned
184	2026-10-07	Claude	Offer the variant and shade from the catalogue, not the price matrix
185	2026-10-07	Claude	Price BF Film per panel, so it can be sold at all
186	2026-10-10	Claude	Redesign the dashboard around what people come to it to do
187	2026-10-10	Claude	Give every screen the dashboard's design language, from one stylesheet
188	2026-10-10	Sean Esperancilla	Make the vehicle checklist a walkaround someone can actually do

B Appendix B — Repository Attribution

Commit authorship as recorded by Git. The figures are stated plainly because the division of labour is itself a fact about how the system was built, and because an engineering report that obscured it would be describing a different project.

Author	Commits	Share
Claude	152	80.9%
Sean	26	13.8%
BENJO LOPEZ	6	3.2%
SeanEsperancilla	3	1.6%
Sean Esperancilla	1	0.5%

The rows above are exactly what Git recorded and are not merged. ‘Sean’, ‘SeanEsperancilla’ and ‘Sean Esperancilla’ are one person committing under three configured identities, which together account for 30 commits (16.0%).

The human authors set the requirements, supplied every price, measurement and business rule in the system, built the initial scaffolding and the first screens, and accepted or rejected each change. The shop’s own price list, the measured glass sheet of section 7.7, the payment practices of 7.2 and 7.3, the plate-format rule and the decision that the database schema was not to be altered all originate with them. Commits authored by Claude were made under that direction and against those constraints, in a working session with the repository owner, and each carries a session reference in its trailer.

Where this report states that a figure was derived rather than supplied — the BF Film markups of section 11.4, the hatchback cut sizes of 14.1 — it is because no human source for it existed, and those are the only such figures in the system.

End of report.